SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q3 – Device-Status Register with Bitwise Operators

Predicted register values (before running)

No.	Statement in main()	Predicted value (binary)	Decimal
1	reg = 0	00000000	0
2	switch_on(reg, LIGHT)	00000001	1
3	switch_on(reg, AC)	00000101	5
4	toggle(reg, FAN)	00000111	7
5	switch_off(reg, LIGHT)	00000110	6

The program prints the register after statements 3 and 5, so the output is 00000101 and 00000110, followed by AC is ON, PUMP is OFF.

Complete the Code – TODO 1 to TODO 4

```
unsigned char switch_on(unsigned char reg, int dev)  { return reg | (1u << dev); }
unsigned char switch_off(unsigned char reg, int dev) { return reg & ~(1u << dev); }
unsigned char toggle(unsigned char reg, int dev)     { return reg ^ (1u << dev); }
int is_on(unsigned char reg, int dev)                { return (reg >> dev) & 1u; }
/* counting: Kernighan's trick, each pass clears the lowest set bit */
int count_on(unsigned char reg){ int c = 0; while (reg) { reg &= reg - 1; c++; } return c; }
```

Output:

```
00000101
00000110
AC is ON, PUMP is OFF
devices on = 2
```

Programiz
C Online Compiler

main.c

Clear Run ▶

```c
#include <stdio.h>

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
    return (reg >> dev) & 1u;
}

```

Output

```
00000101
00000110
AC is ON, PUMP is OFF
devices on = 2
```

Inquiry

- **reg & 1 << dev** – << has higher precedence than &, so it means reg & (1 << dev): it tests bit dev. For reg = 00000110, dev = 2 it gives 4 (non-zero = ON).
- **(reg & 1) << dev** – first isolates bit 0 (LIGHT), then moves that bit to position dev. For the same reg it gives 0 because LIGHT is off. It is a different question, so always add brackets.
- **signed char with bit 7 set:** 0x80 is −128; it is promoted to int with sign extension (0xFFFFFF80). Measured: sc >> 7 = −1 (arithmetic shift, implementation-defined for negatives) versus 1 for unsigned char; sc == 0x80 is false because −128 ≠ 128 (gcc warns -Wtype-limits). Use unsigned char for bit registers.

Alternative Concept and Complexity Analysis

```
char state[8];
state[dev] = 1;      /* on  */
state[dev] = 0;      /* off */
state[dev] ^= 1;     /* toggle */
return state[dev];   /* test */
int count=0; for (int i=0;i<8;i++) count += state[i];
```

Every single-device operation is O(1) in both designs. Counting the devices that are ON: the array needs a loop of 8 steps, O(d) for d devices; the bit register needs at most one pass per set bit (Kernighan), or a 256-entry lookup table in O(1). Space for 1,000,000 homes: 1 byte per home = 1 MB with the register versus 8 bytes per home = 8 MB with the array (8× more). The array is simpler and allows storing more than ON/OFF per device (e.g. a dimmer level), but the register also lets one AND/OR change many devices at once.